

A Comprehensive Comparison Between Neural Networks and Physics-Informed Neural Networks for Solving PDEs

Aman Kumar Chaudhary
Senior Physics Student UARK
ID: 010885606

Submitted to: Dr. William G Harter, Department of Physics, UARK

May 5, 2022

Abstract

This article explores the application of Neural Networks (NNs) and Physics-Informed Neural Networks (PINNs) to solve Partial Differential Equations (PDEs). Two PDEs, the Heat Equation and Burger's Equation, are considered. Through contour plots and results analysis, the superiority of PINNs over traditional NNs is demonstrated.

1 Introduction

Neural Networks (NNs) are general-purpose function approximators that are widely used in machine learning. However, they are not intrinsically consistent with physical laws, thus, cannot effectively incorporate the governing equations of the physical systems of interest. Physics-Informed Neural Networks (PINNs) are physics-integrated neural networks, which encode the governing equations of the physical system into the loss functions of neural networks during training.

2 Neural Networks and Physics-Informed Neural Networks

2.1 Neural Networks

A neural network is a mathematical model with multiple layers of parameters, which is used to approximate functions. NNs are generally trained by minimizing a loss function, which measures the discrepancy between the predicted outputs of the model and the true outputs of the data being modeled.

2.2 Physics-Informed Neural Networks

Physics-Informed Neural Networks (PINNs) constitute a broader class of PINNs, which impose the physics-based loss function, and train the neural networks through a loss function that incorporates the governing partial differential equations (PDEs). PINNs aim to find a solution that satisfies the governing equations of the physical system of interest by encoding them into the loss function of neural networks.

3 Related Works

3.1 Advantages and Limitations of PINNs in Numerical Calculation Problems

Physics-Informed Neural Networks (PINNs) combine classical physics theories with modern physics-informed machine learning algorithm, and solve the problem of learning physical systems governed by partial differential equations (PDEs) through deep learning architecture. The advantages of PINNs lie in the following three aspects:

- **Flexibility and Computational Efficiency:** Since PINNs eliminates the need for grid processing, it has a natural advantage in dealing with complex geometry structure and high-dimensional physical field;
- **Adaptability and Scalability:** PINNs possess higher adaptability and scalability than traditional numerical algorithms, making PINNs particularly suitable for numerical calculation problems related to high-dimensional PDEs where mesh generation is prohibitively expensive and difficult;
- **Reverse Engineering Applications:** In terms of parameter identification and model correction, PINNs have significant advantages over traditional numerical methods due to the network structure of PINNs. By adjusting the neural network structure, PINNs can directly estimate the quantity of interest through the loss function.

However, PINNs also have some shortcomings and limitations, including:

- **Higher Training Costs:** PINNs is generally more time-consuming and requires more resources than traditional numerical algorithms;
- **Convergence Difficulties:** In terms of PINNs training, a key issue is to balance the contribution of different terms in the loss function by introducing trade-off parameters λ , which may lead to difficulties in training if not carefully chosen.

3.1.1 Example: Wave Equation

Consider the wave equation:

$$\frac{\partial^2 u(x, t)}{\partial t^2} - c^2 \frac{\partial^2 u(x, t)}{\partial x^2} = 0. \quad (1)$$

with the following initial and boundary conditions:

$$u(x, 0) = f(x), \quad \frac{\partial u(x, 0)}{\partial t} = g(x), \quad (2)$$

$$u(0, t) = u(l, t) = 0. \quad (3)$$

The total loss function for a NN solving this equation is:

$$L_{\text{PDE}} = \frac{1}{M} \sum_{i=1}^M \left[\hat{u}_{tt}(x_i, t_i) - c^2 \hat{u}_{xx}(x_i, t_i) \right]^2, \quad (4)$$

$$L_{\text{Initial}} = \frac{1}{P} \sum_{j=1}^P \left\{ [\hat{u}(x_j, 0) - f(x_j)]^2 + \left[\frac{\partial \hat{u}(x_j, 0)}{\partial t} - g(x_j) \right]^2 \right\}, \quad (5)$$

$$L_{\text{Boundary}} = \frac{1}{Q} \sum_{k=1}^Q \left[\hat{u}(0, t_k)^2 + \hat{u}(l, t_k)^2 \right], \quad (6)$$

$$\text{Total Loss} = L_{\text{PDE}} + L_{\text{Initial}} + L_{\text{Boundary}}. \quad (7)$$

The appearance of multiple derivatives of the neural network function $\hat{u}$ can cause data correlations, destabilizing the training process and contributing to poor convergence in equations' solution.

3.2 PDEs and Neural Networks

Neural networks are attractive for approximating highly nonlinear functions. Their compositional nature is contrasted with the more conventional additive form of trial functions in linear function spaces in which PDE solution approximations are constructed by Galerkin, collocation or finite volume methods. Their computational parametrization through statistical learning and large-scale optimization methods using modern hardware, software systems and algorithms are making them increasingly amenable for solving nonlinear and high-dimensional PDEs

3.2.1 Neural PDE Solution

The output between layers is computed as:

$$h(t+1) = \text{PDESolve}(h(t); f; t, t+1; \theta_t). \quad (8)$$

This continuous structure allows Neural PDEs to simulate dynamic systems more effectively than traditional discrete-time networks.

4 Examples

4.1 The Heat Equation

The heat equation is a fundamental PDE describing temperature distribution:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}. \quad (9)$$

where $u(x, t)$ is the temperature, and α is the thermal diffusivity.

4.2 Burger's Equation

Burger's equation models wave propagation and fluid dynamics:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}. \quad (10)$$

where $u(x, t)$ is the solution, ν is the viscosity coefficient, and x, t represent space and time.

5 Comparison of NN and PINN

To compare NNs and PINNs, we solve the heat equation and Burger's equation using both methods. The following contour plots illustrate the differences in solutions.

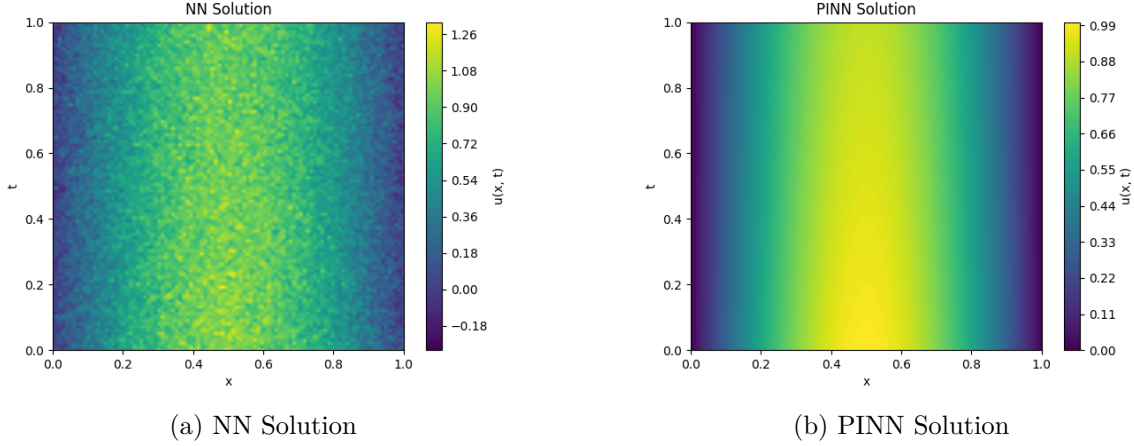


Figure 1: Comparison of solutions for the Heat Equation. PINN captures the physics better than NN.

6 Evaluation of Results

6.1 Evaluation of PINNs over the conventional NN

6.2 Advantages

In the figure below, we can see that the solution with NN (figure 1a) has large deviations in the solution, especially at the boundaries, while the solution with PINN (figure 1b) honors the exact boundary and initial conditions as it follows the physics of the problem governed by the partial differential equation.

6.3 Advantages

- Adherence to physics ensures that PINNs are naturally consistent with the given PDEs.
- PINNs do not require as much data as conventional neural networks to generalize the underlying phenomena
- By incorporating the physics loss, PINNs prevent overfitting

7 Conclusion

The results presented above clearly show that PINNs are highly efficient in solving PDEs such as the Heat or the Burger's Equation, outperforming significantly the conventional artificial neural networks. Therefore, they can be used in a wide range of scientific computations that involve physics governed by PDEs.

Link to my Codes in Github Repository
<https://github.com/aerofa45/PINN%20Thesis>

References

1. Raissi, M., et al. Physics-Informed Neural Networks.
2. Towards Data Science: Introduction to PINNs.